

Quelques informations sur notre projet Java

Gaëtan Houllier, Matteo Quintaneiro

October 30, 2025

1 Les instances d'entrée

Les instances d'entrée sont réparties en deux catégories, **Small** et **Large** :

- **Small** : comprend 6, 8, 10 et 12 tâches pour 2, 3, 4 et 5 machines, respectivement.
- **Large** : comprend 25, 50, 100, 150 et 200 tâches pour 10, 15, 20, 25 et 30 machines, respectivement.

Chaque instance est organisée dans un dossier contenant :

- Un fichier *task.txt*, dont la première ligne indique le nombre de tâches n et le nombre de machines m . Les lignes suivantes constituent une matrice de taille $n \times m$, où chaque ligne de la matrice, équivalent à une tâche, correspond aux temps d'exécution de cette tâche sur les différentes machines.
- Un sous-dossier *machine*, contenant m fichiers **machine_[numéro machine].txt**. Chaque fichier représente une matrice de taille $n \times n$ indiquant, pour cette machine, le temps de setup lorsque l'on passe de la tâche de la ligne i à la tâche de la colonne j .

Les valeurs des matrices ont été générées aléatoirement à partir des intervalles $\{1-9\}$, $\{1-49\}$, $\{1-99\}$ et $\{1-124\}$. Pour chaque catégorie, 10 instances ont été créées. Au total, quatre paramètres principaux définissent les instances : le nombre de tâches, le nombre de machines, l'intervalle des temps de setup et le nombre d'instances par configuration. On se retrouve ainsi avec $4 \times 4 \times 4 \times 10 = 640$ instances de petite taille et $5 \times 5 \times 4 \times 10 = 1000$ instances de grande taille.

2 Organisation du code

Le code est organisé en 5 classes principales : **Solution.java**, **Machine.java**, **Data.java**, **Main.java** et **GanttSimulator.java**.

- **Solution.java** : implémente l'objet solution, avec un vecteur de machines.
- **Data.java** : lit et récupère les données d'entrée avant de résoudre le problème.
- **Machine.java** : représente une machine et contient un vecteur des tâches qu'elle doit exécuter, ainsi que le temps total de complétion.
- **Main.java** : contient l'algorithme **Late Acceptance Hill Climbing**, utilisé pour générer des solutions optimisées au problème étudié.

- `GanttSimulator.java` : fournit une interface graphique permettant de visualiser les planings générés sous forme de diagrammes de Gantt. Il est possible de charger ses propres fichiers ou de générer un problème aléatoire et de suivre l'évolution de l'algorithme.

Le diagramme de classes suivant montre les relations entre les différentes classes. Le fichier `GanttSimulator` n'étant pas nécessaire au fonctionnement de l'algorithme n'a pas été représenté.

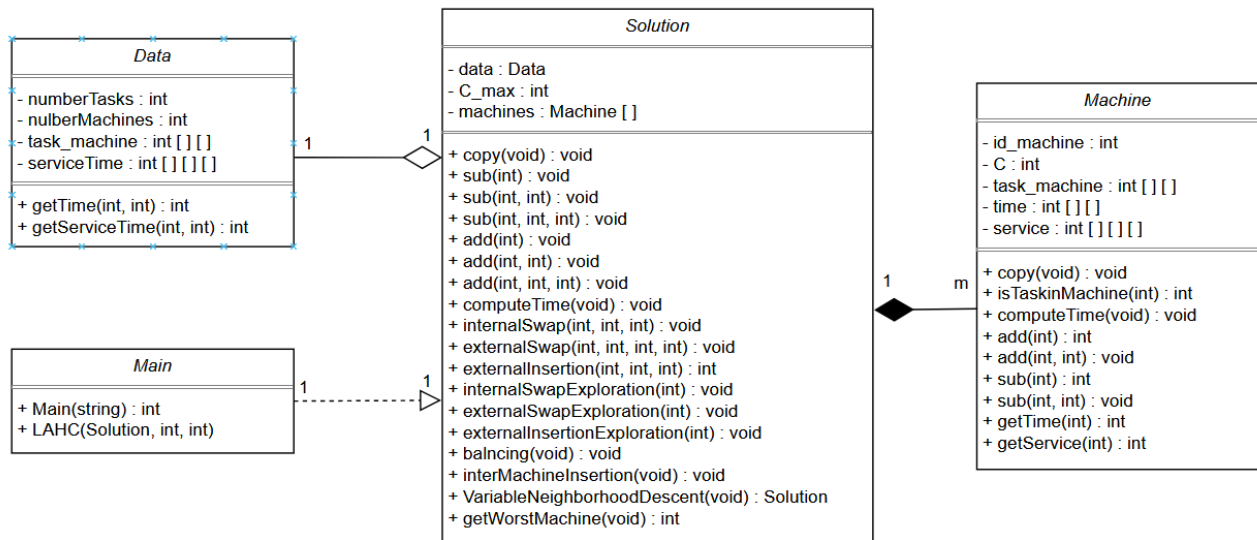


Figure 1: Diagramme UML de notre projet java

3 Compilation et Lancement du projet

Les commandes ci-dessous permettent de compiler et de lancer le projet sous **Java 17** selon le système d'exploitation utilisé en se plaçant dans le répertoire du projet. (Vérifier la ligne de commande si copier coller car elle peut être modifiée).

3.1 Windows (PowerShell / CMD)

3.1.1 Compilation

```
javac --module-path "lib\openjfx-17.0.17_windows-x64_bin-sdk\javafx-
  sdk-17.0.17\lib" --add-modules javafx.controls,javafx.fxml,javafx.
  graphics GanttSimulator.java
```

3.1.2 Exécution

```
java --module-path "lib\openjfx-17.0.17_windows-x64_bin-sdk\javafx-sdk
  -17.0.17\lib" --add-modules javafx.controls,javafx.fxml,javafx.
  graphics GanttSimulator
```

Remarque : Exécuter ces commandes dans le répertoire racine du projet.

Pour macOS et Linux, je n'ai pas pu laisser les packages car ils étaient trop gros dans le zip mais ils peuvent être retéléchargés sur <https://gluonhq.com/products/javafx/> et placer des le dossier lib puis lancer ces deux commandes:

3.2 macOS (Terminal)

3.2.1 Compilation

```
javac --module-path "lib/openjfx-17.0.17_macos-x64_bin-sdk/javafx-sdk-17.0.17/lib" --add-modules javafx.controls,javafx.fxml,javafx.graphics GanttSimulator.java
```

3.2.2 Exécution

```
java --module-path "lib/openjfx-17.0.17_macos-x64_bin-sdk/javafx-sdk-17.0.17/lib" --add-modules javafx.controls,javafx.fxml,javafx.graphics GanttSimulator
```

3.3 Linux (Terminal / WSL)

3.3.1 Compilation

```
javac --module-path "lib/openjfx-17.0.17_linux-x64_bin-sdk/javafx-sdk-17.0.17/lib" --add-modules javafx.controls,javafx.fxml,javafx.graphics GanttSimulator.java
```

3.3.2 Exécution

```
java --module-path "lib/openjfx-17.0.17_linux-x64_bin-sdk/javafx-sdk-17.0.17/lib" --add-modules javafx.controls,javafx.fxml,javafx.graphics GanttSimulator
```

Il est possible de générer un problème aléatoire en donnant le nombre de machines et le nombre de tâches. On peut également poser un problème soi-même en fournissant le fichier des tâches et le répertoire des machines. Pour lancer les instances qui ont été générées, lancer le bash qui a été prévu à cet effet dans le terminal : `./instances.bash`

4 Résultats

4.1 Premiers résultats sur nos instances

On peut voir que, sur nos deux ensembles d'instances, le temps d'exécution de notre algorithme décroît de plus en plus rapidement lorsque le nombre de machines augmente, mais croît de manière exponentielle lorsque le nombre de tâches augmente. La fonction objectif diminue également de plus en plus vite lorsque le nombre de machines augmente, mais croît de manière linéaire lorsque le nombre de tâches augmente.

4.2 Comparaison à la solution optimale

Afin d'évaluer l'efficacité de notre solution, et n'ayant pas pu récupérer les instances ni les résultats de l'article, nous avons décidé, après un échange par mail avec Mourad Terzi, d'installer le solveur Gurobi. Le temps de calcul de ce solveur s'est révélé très long, même pour des instances de petite taille (par exemple, pour 12 tâches, il nécessite environ 2000 secondes pour

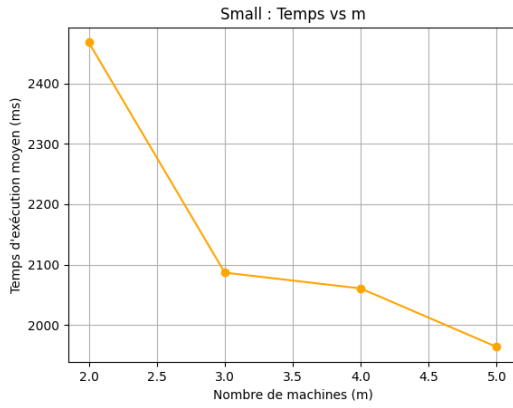


Figure 2: Temps d'exécution moyen en fonction du nombre de machines (m) pour les instances *small*.

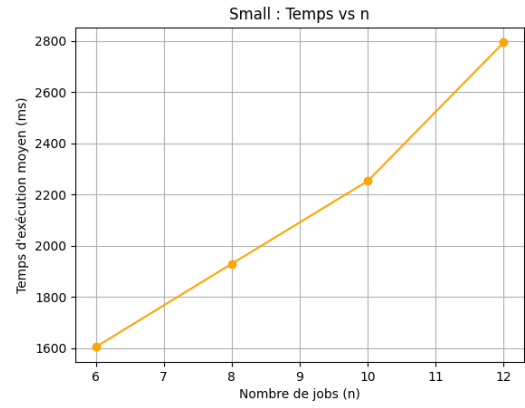


Figure 3: Temps d'exécution moyen en fonction du nombre de tâches (n) pour les instances *small*.

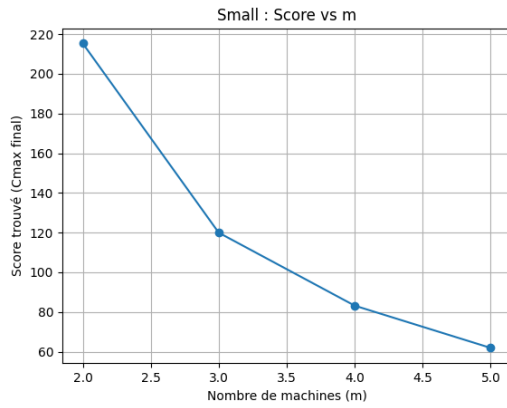


Figure 4: Score final (C_{max}) en fonction du nombre de machines (m) pour les instances *small*.

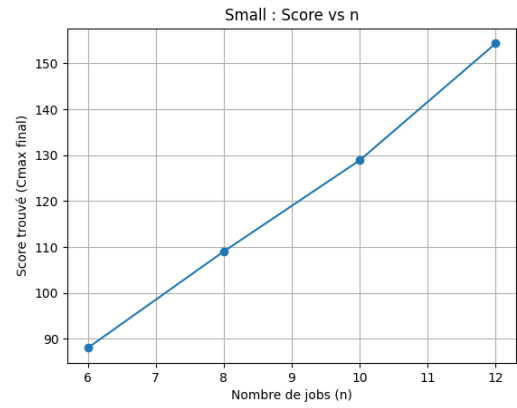


Figure 5: Score final (C_{max}) en fonction du nombre de tâches (n) pour les instances *small*.

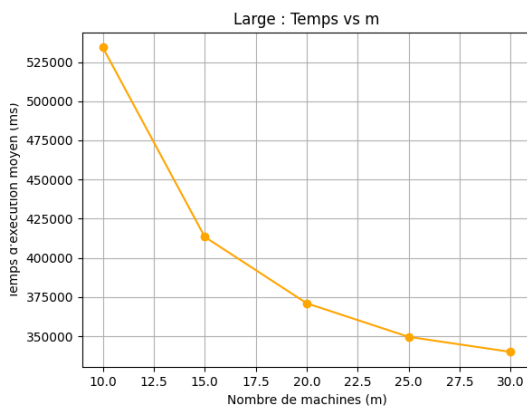


Figure 6: Temps d'exécution moyen en fonction du nombre de machines (m) pour les instances *large*.

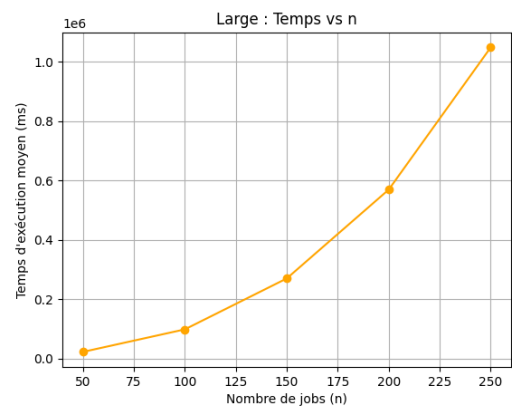


Figure 7: Temps d'exécution moyen en fonction du nombre de tâches (n) pour les instances *large*.

résoudre l'instance). Nous présentons donc ici une partie des résultats obtenus. Lors de la soutenance, il sera tenté de présenter des résultats plus complets.

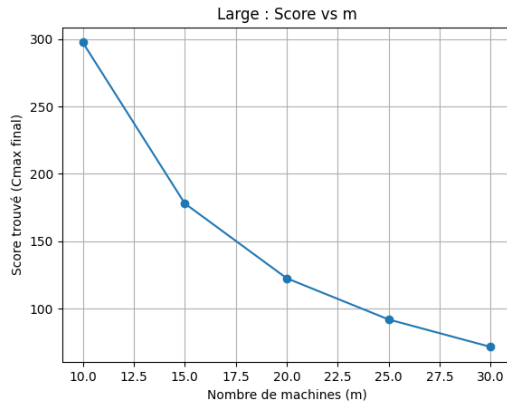


Figure 8: Score final (C_{max}) en fonction du nombre de machines (m) pour les instances *large*.

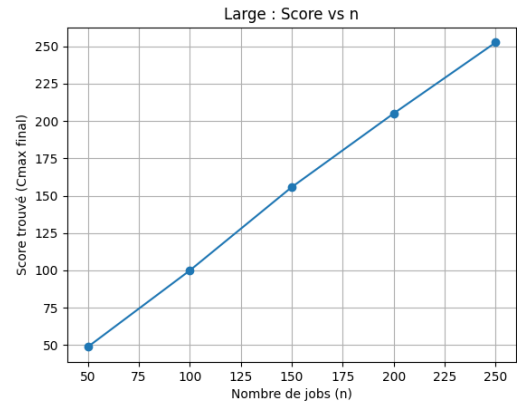


Figure 9: Score final (C_{max}) en fonction du nombre de tâches (n) pour les instances *large*.

Classe	LAHC	T(s)	Gurobi	RPD
6x2	158,55	1,7	155,4	2,03 %
6x3	84,9	1,6	84,325	0,68 %
6x4	63,475	1,5	63,45	0,04 %
6x5	45,175	1,4	45,175	0,00 %
8x2	196,1	2	189,225	3,63 %
8x3	114,6	1,9	114,45	0,13 %
8x4	71,4	1,9	71,375	0,04 %
8x5	54,05	1,7	54,025	0,05 %
10x2	224,025	2,6	219,8	1,92 %
10x3	127,65	2,1	126,375	1,01 %
10x4	94,2	2,2	94,1	0,11 %
10x5	69,75	2,1	69,75	0,00 %

Figure 10: Comparaison sur les petites instances

On observe que, de manière générale, notre algorithme offre d'excellentes performances et parvient à des résultats très proches de la solution optimale.